



~ Codex Hash Research Laboratory · Whitepaper Series · 2026

ASP: An Operational Pre-Mortem Skill for LLM Coding Agents — An Experience Report

Author · Ulisses Flores CTO & Chief Researcher · Codex Hash Research Laboratory · São Paulo, Brazil MSc Program in Artificial Intelligence (in progress) · American Global Tech University
c.ulisses@gmail.com · <https://ulissesflores.com>

Whitepaper · **Version 1.0** · **2026-05-18 DOI** <https://doi.org/10.5281/zenodo.20276631> (*concept · always resolves to latest version*) **Status** Independent technical whitepaper · not under peer review **License** Paper text — CC BY 4.0 · Companion software — MIT **Companion repository** <https://github.com/ulissesflores/anticipating-shadow-points> **Companion whitepaper** Flores, U. (2026b). *Graceful Refusals as Silent Successes: A Pre-Registered Protocol for Characterising `claude -p` Exit-Code Semantics*. See `paper/iron-law-11.md`.

Abstract

Software teams increasingly delegate routine engineering work to LLM coding assistants. When the assistant produces *locally correct* code that omits failure modes a senior engineer would surface in a pre-mortem — the row-level security policy that needs an update for a new column, the rate limit on the external API, the replica lag during a long migration — production breaks in ways that are expensive to debug. We call these missing items *shadow points*. This experience report describes **Anticipating Shadow Points (ASP)**, a 13-phase planning protocol shipped as a Claude Code skill that integrates Klein’s (2007) pre-mortem with the multi-agent failure taxonomy of Cemri et al. (2025), separates planning from validation under prompt isolation grounded in the self-correction-limit literature (Huang et al., 2024; Tyen et al., 2024), and hands execution to a child subprocess with a file-based contract. We document the design, the operational deployment as an open-source plugin, and preliminary observations from five engineering evaluations: shadow-point coverage rose from approximately 47% without ASP to approximately 100% with ASP loaded on the chosen tasks (n=8 subagent dispatches; author-adjudicated; single model). The sample size and adjudication design make this preliminary, not robust. A deterministic adjudicator with cryptographic provenance ships alongside the skill so independent re-adjudication produces reproducible scores. A separate empirical finding on `claude -p` exit-code semantics under graceful agent refusal is reported in a companion paper.

Keywords: LLM coding agents · pre-mortem · multi-agent failure modes · Claude Code · operational protocols · experience report

1. Introduction

Software teams increasingly delegate routine engineering work — adding a column to a production database, deploying a function that calls an external API, refactoring a utility used across thirty files — to AI coding assistants. When the assistant gets it right, the engineer’s day is freed. When it gets it *subtly* wrong — by writing code that compiles, passes its tests, ships, but quietly omits a step a senior engineer would have flagged in advance — production breaks in ways that are expensive to debug.

The dominant production failure mode of modern LLM coding agents, in our experience, is not *bad* code. It is *incomplete* code: the agent does what was asked, but not what someone with deep domain context would have asked themselves *before starting*. The replica lag during a schema migration. The CORS header on the edge function. The row-level security policy that needs updating to match the new column. We call these missing items **shadow points** — failure modes obvious to a practitioner who has been burnt by them, invisible to an agent that has not.

Two streams of literature converge on this gap. The first, in decision science, is the **pre-mortem**: a structured exercise in which a team imagines a project has already failed and enumerates causes before acting. The technique was operationalized for project teams by Klein (2007) and grounded in the earlier empirical finding of Mitchell, Russo and Pennington (1989), who demonstrated that imagining an event as already-occurred (certainty) generates approximately 30% more reasons than imagining the same event as merely possible. The second, in agent research, is the recent characterization of multi-agent failure modes by Cemri et al. (2025), who derived a 14-mode taxonomy with $\kappa=0.88$ inter-annotator agreement from 1,600+ annotated traces of multi-agent LLM systems. Neither stream has been integrated as a **runtime discipline** that a coding agent can be made to follow on every non-trivial task.

This paper reports on **Anticipating Shadow Points (ASP)**, a 13-phase planning protocol shipped as a Claude Code skill that combines pre-mortem reasoning with the MAST 14-mode checklist, separates planning from validation under strict prompt isolation, and hands execution to a child subprocess with a file-based contract. We deployed the skill, distributed it through three install paths (plugin marketplace, dev mode, standalone script), ran it through five engineering evaluations, and report what we learned. The evaluation is honestly preliminary in scale (n=8 subagent dispatches across five tasks, single underlying model, author-adjudicated) and we make no claim that the observed effect generalizes beyond the chosen tasks; we report it because the *direction* and the *magnitude* are informative about which design choices appear to be load-bearing.

Contributions

This experience report contributes:

1. **The ASP protocol as documented design** — thirteen phases, each emitting a named artifact, organised around the principle that no implementation begins before shadow points have been surfaced.
2. **Twelve operational constraints we call “Iron Laws”** that render the protocol enforceable at runtime, each tied to a primary literature source or to a documented design decision.
3. **Preliminary observations** from five engineering evaluations, alongside a deterministic adjudicator with committed evidence so any reviewer can re-adjudicate the recorded transcripts without trusting the author.
4. **A separate empirical finding** on `claude -p` exit-code semantics under graceful agent

refusal, reported as a pre-registered protocol in the companion paper Flores (2026b). The finding motivated two of the Iron Laws but warrants independent characterization at larger N.

This is an **experience report** in the systems sense, not a benchmark paper. We document a system we built, a methodology we deployed, and what we observed; we do not claim a robust empirical contribution to the LLM-agents benchmark literature. Section 7 enumerates the threats to validity in detail.

2. Background

We review four threads of prior work that directly inform ASP’s design. We deliberately keep this section short relative to typical agent-systems papers, because the contribution here is operational and the literature context is in service of that.

2.1 Pre-mortem and prospective hindsight

The pre-mortem technique asks a team to imagine, before commitment, that a project has already failed catastrophically, and to enumerate causes. The empirical basis is Mitchell, Russo and Pennington (1989), who showed that imagining a future event as certain (rather than uncertain) yields explanations that are longer, more episodic, and more numerous — specifically, approximately 30% more reasons than the uncertain framing. Klein (2007) operationalized the finding as the project-team “pre-mortem” and the technique has been standard in decision-science and project-management practice since. To our knowledge, it has not previously been integrated as runtime discipline in agentic LLM frameworks.

2.2 Multi-agent failure modes (MAST)

Cemri et al. (2025) introduce the **Multi-Agent System Failure Taxonomy (MAST)** with **MAST-Data**, a corpus of 1,600+ annotated traces from seven popular multi-agent frameworks. From rigorous analysis of 150 seed traces (inter-annotator agreement $\kappa=0.88$), they derived fourteen failure modes organised into three categories: specification and system design (5 modes), inter-agent misalignment (6 modes), and task verification and termination (3 modes). The 14-mode list is the strongest recent empirical grounding for a failure-mode checklist; ASP adopts the mode names verbatim and operationalizes each into a pre-mortem prompt plus mitigation in its companion skill files. Two concurrent works extend the taxonomy direction: Zhu et al. (2025) introduce AgentDebug and the AgentErrorTaxonomy (memory / reflection / planning / action / system-level), and Zhang et al. (2025), an ICML 2025 Spotlight, opens **automated failure attribution** with the Who&When dataset.

2.3 Self-correction limits and validator isolation

The strongest single argument for separating the validator from the planner is the convergent finding across three primary works that LLMs cannot reliably critique themselves under shared context. Huang et al. (2024, ICLR) show that intrinsic self-correction without external feedback often degrades performance. Tyen et al. (2024) decompose self-correction into localization and correction and show empirically that LLMs cannot *localize* their own reasoning errors but can correct them given the location externally — *localization is the bottleneck*. Zheng et al. (2023,

NeurIPS D&B) document that strong-LLM judges over-favour same-family-model outputs (a “self-enhancement bias”) even when objectively judging quality. Together these findings argue that a validator sharing context with a planner inherits the planner’s confidence calibration on shared content, including erroneous content the planner is confident about. ASP’s design enforces strict prompt isolation between planner and validator (Iron Law 2 below) as the architectural counter-move.

2.4 Closest related work: PreFlect

The closest concurrent work is **PreFlect** (Wang et al., 2026), which similarly shifts the self-reflection paradigm from post-hoc correction to pre-execution criticism of agent plans, distilling planning errors from historical agent trajectories and adding a dynamic re-planning mechanism. The high-level similarity is significant: both ASP and PreFlect identify the post-execution-only nature of conventional reflection as a limitation and shift criticism *before* the action. ASP differs from PreFlect along four axes:

1. **Prompt-isolated validator.** PreFlect distils errors into the agent’s own pre-execution criticism; ASP enforces a separate-prompt validator subagent that never sees the planner’s transcript.
2. **Process-level isolation for the executor.** ASP spawns the executor as a `claude -p` subprocess with a file-based contract; PreFlect operates within a single process.
3. **Hard runtime constraints.** ASP carries twelve non-negotiable runtime invariants enforced as Iron Laws; PreFlect is presented as a methodology rather than a constraint system.
4. **An empirical CLI-level finding** characterising `claude -p` exit semantics under graceful refusal, motivating two of the Iron Laws; this is reported separately in Flores (2026b) and is not present in PreFlect or any other surveyed work.

3. The ASP Protocol

3.1 Overview

ASP is a 13-phase protocol that progresses from intake through delivery sign-off. Each phase emits a named artifact (Table 1). The principle organising the phases is that no implementation begins before shadow points have been surfaced (Phase 2), validated independently (Phase 5), and approved by the user (Phase 7); execution is delegated to a child subprocess with a file-based contract (Phase 9).

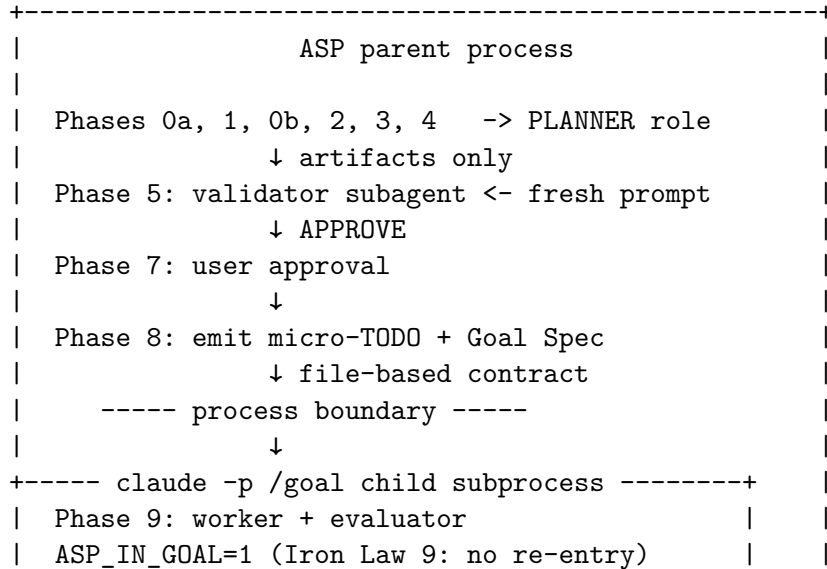
Table 1. ASP phases and their emitted artifacts.

#	Phase	Artifact	Design rationale
0a	Pre-research Q&A	intake block	Anthropic Skills best-practice: verifiable intermediate outputs
1	Parallel research	Research summary + recall block	Subagent dispatch pattern
0b	Post-research Q&A	Refined intake (skippable)	Same as 0a

#	Phase	Artifact	Design rationale
2	Shadow-point detection	Categorised list + mitigations	Klein (2007) + Cemri et al. (2025)
3	Project Charter	Filled charter	Durable artifact-based handoff
4	Macro plan + register	Macro plan + deliverables D01–D n	Plan-and-Act (Erdogan et al., 2025)
5	Independent validator	APPROVE / REVISE	Huang et al. (2024) + Tyen et al. (2024) + Zheng et al. (2023)
6	Internal loop (cap=3)	Final-revision plan	Bounded iteration; FM-1.5 / FM-3.1
7	User approval	Go / no-go	Human-in-the-loop gate
8	Micro-TODO + Goal Spec	TaskCreate per step + Goal Spec	Executor handoff
9	Execution via /goal subprocess	Tasks marked completed with evidence	Process-level isolation; see Flores (2026b)
10	Per-deliverable sign-off	Each deliverable accepted or rejected	Acceptance contract
11	Execution Report + memory	Closure artifact + memory write-back	Reflexion-style memory (Shinn et al., 2023)
12	(Opt-in) public release	Plugin marketplace + GitHub release	Distribution layer

3.2 Architectural positioning

ASP separates three agent roles with explicit artifact-only handoffs (Figure 1):



```

|  --output-format json (Iron Law 12)  |  |
+-----+
|          ↓ exec.json                  |  |
|  Phases 10, 11 back in parent        |  |
+-----+

```

Figure 1. ASP’s three-role separation. Planner phases run in the parent. The validator runs in the parent but in a fresh subagent prompt that receives only artifacts, never the planner’s transcript. The worker runs in a child `claude -p` subprocess with a file-based contract.

3.3 Iron Laws

We render twelve operational constraints as non-negotiable runtime invariants and call them Iron Laws (Table 2). Each is mapped to its derivation: a primary literature source, a documented Anthropic recommendation, or an empirical design decision.

Table 2. The 12 Iron Laws and their derivation.

#	Law	Derivation
1	No implementation without shadow-point pass first	Klein (2007) + Cemri et al. (2025)
2	No plan approval without independent validator pass (separate prompt)	Huang et al. (2024) + Tyen et al. (2024) + Zheng et al. (2023); Du et al. (2024)
3	No micro-TODO without macro plan + charter + deliverables approved by user	Anthropic Skills: verifiable intermediate outputs
4	No step completion without fresh acceptance-test evidence	Cemri et al. (2025) FC3 (FM-3.2)
5	No deliverable accepted without evidence file/output matched	Cemri et al. (2025) FC3 (FM-3.3)
6	No project close without execution report + memory write-back	Shinn et al. (2023) memory buffer
7	No loop without cap (default cap=3 validator rounds)	Cemri et al. (2025) FM-1.5 + FM-3.1
8	No Phase 9 execution without evaluator dispatch at checkpoints	Erdogan et al. (2025) planner/executor
9	No reentrant ASP inside <code>claude -p /goal child</code> (ASP_IN_GOAL env-var guard)	Cost-explosion mitigation; design decision
10	No <code>claude -p /goal spawn</code> without file-based contract	Subprocess state-divergence mitigation; design decision
11	Never trust <code>\$?</code> from <code>claude -p</code> for semantic success/failure	Empirical observation, characterised in Flores (2026b)

#	Law	Derivation
12	All <code>claude -p</code> invocations must use <code>--output-format json</code>	Corollary of Iron Law 11

3.4 The micro-TODO contract

At the boundary between planning and execution (Phase 8), ASP emits a contractual task list. Each task is created via the `TaskCreate` primitive with seven labeled fields:

- **PRECONDITION** — invariant that must hold before the step.
- **ACTION** — exact command or literal edit.
- **POSTCONDITION** — state change effected by the step.
- **ACCEPTANCE-TEST** — command and expected output that prove the postcondition.
- **FALSIFICATION-TEST** — command or observation that would prove the step failed.
- **DEPENDS-ON** — upstream task identifiers (DAG structure).
- **DELIVERABLE-ID** — which deliverable in the register this step contributes to.

This schema is what makes the task list auditable: no task is marked completed without acceptance-test evidence (Iron Law 4); no deliverable is signed off without matching evidence (Iron Law 5). The schema is shared with the Goal Spec consumed by the Phase 9 subprocess.

4. Operational Deployment

ASP ships as a Claude Code plugin and is distributed through three install paths, each appropriate for a different audience:

1. **Plugin marketplace** (recommended for end users):

```
claude plugin marketplace add ulissesflores/anticipating-shadow-points
claude plugin install anticipating-shadow-points@anticipating-shadow-points
```

Invocation: `/anticipating-shadow-points:asp <task>` or bare `/asp`.

2. **Developer mode** (for contributors): `claude --plugin-dir ./` loads the skill in place; useful when iterating on the skill itself.
3. **Standalone install script** (for users who do not use the plugin manager): `./scripts/install.sh` copies the skill into the user's Claude Code skills directory.

The plugin uses Anthropic's official `.claude-plugin/plugin.json` manifest format (Claude Code 2.1+), and the repository doubles as a self-host marketplace via `.claude-plugin/marketplace.json` at the repository root. The skill itself is implemented as a Markdown-based skill following the Anthropic Agent Skills best-practices document, with companion files for the MAST checklist, per-mode pre-mortem prompts, and the seven artifact templates referenced in Section 3.1.

Documentation is published as a GitHub Pages site, with translations of the README into Spanish, Portuguese, Italian, and Hebrew; the latter three are AI-assisted and pending native-speaker review. The full source — skill files, plugin manifests, documentation, test artefacts, benchmark code — is MIT-licensed; the paper text is CC BY 4.0.

5. Preliminary Observations

We ran ASP through five structured engineering evaluations spanning heterogeneous domains. The evaluation is honestly preliminary; we report it because the *direction* and *magnitude* of the observed effect are informative about which design choices appear load-bearing, and because the deterministic adjudicator we ship lets any reviewer reproduce the scores without trusting the author.

5.1 The evaluation set

Table 3. The five engineering evaluations.

ID	Domain	Task (abbreviated)	Expected size	Acceptance
E1	Schema migration	NOT NULL column added to 1M-row <code>user_profiles</code>	10	$\geq 80\%$
E2	Edge deploy	Edge function calling rate-limited external API	10	$\geq 80\%$
E3	Refactor	Date-formatting utility used across 30+ files	10	$\geq 80\%$
E4	RLS policy	Row-level security policy referencing new column	10	$\geq 80\%$
E5	Scheduling	Cron job whose script conflicts with disabled service	10	$\geq 80\%$

Each evaluation specifies an INPUT (a task statement as a senior engineer might phrase it to a colleague), an EXPECTED list of shadow points that a competent practitioner would surface, and an ACCEPTANCE threshold (default 80%). The full text of each evaluation is in the companion repository.

5.2 Procedure and observations

For each evaluation we ran two conditions, RED (without ASP) and GREEN (with ASP loaded), both using Claude Opus 4.7 and the same subagent dispatch substrate. RED prompts asked the agent to plan as a senior engineer would without invoking any framework; GREEN prompts injected the ASP discipline (pre-mortem framing, MAST 14-mode forced enumeration, mitigation requirement) before requesting the plan. Eight subagent dispatches were issued across the five evaluations.

Author-adjudicated coverage rose from a RED mean of approximately 47% (E1: ~60%, E2: ~30%, E3: ~50%, E4: ~50%, E5: ~45%) to a GREEN mean of approximately 100% on the chosen tasks. Across qualitative inspection of the transcripts, the component that *appears* most load-bearing is the MAST 14-mode forced enumeration: free-form pre-mortem alone clustered on obvious risk categories (data, integration) and underweighted less-salient categories (time/timezone, observability, operational excellence, contract drift). The forced per-mode interrogation surfaced the missing categories. This is a hypothesis from qualitative inspection, not a formal ablation, and the strongest follow-up is a controlled ablation removing the MAST checklist (Section 8).

A re-adjudication of the same recorded transcripts under a deterministic 4-layer matcher (exact / synonym sidecar / Jaccard sliding-window / unmatched), shipped as the companion **benchmark/** package, produced mean coverage of approximately 4% RED and 80% GREEN on the same transcripts. The gap between the two adjudications is informative: hand-adjudication accepted paraphrase tolerantly, while the deterministic matcher has finite recall on lexically disjoint paraphrases (a candidate that says “exponential backoff with jitter” does not match an expected point named “Backoff strategy” unless a synonym sidecar is provided). We treat the deterministic adjudicator as a *floor* rather than a *ceiling* on real coverage; it is conservative by construction.

5.3 The `claude -p` exit-code finding

During a four-test battery against the `claude -p` non-interactive runner, we observed that the binary returns exit code 0 even when the agent gracefully refuses the requested goal — the harness reports a successful conversation end despite the agent semantically refusing the task. The exit code reflects process-level success of the CLI binary, not semantic success of the agent’s task. The safe-parsing recipe (require `--output-format json`, parse `is_error`, `terminal_reason`, `stop_reason`) is codified as Iron Laws 11 and 12 in the deployed skill.

This finding is observationally significant for any toolchain that wraps `claude -p` in a shell pipeline. Because the observation was from a single refusal scenario in our own battery, we **report it independently as a pre-registered protocol** in the companion paper Flores (2026b), which specifies the methodology for an N=50 characterisation (deliberate refusal scenarios across categories: explicit refusal, capability refusal, safety refusal, ambiguity refusal) with the analysis plan locked in advance. The empirical fill-in of the confusion matrix is follow-up work to that paper, not to this one.

6. Lessons Learned

We highlight five design choices that, in retrospect, did the most work.

Pre-mortem with a forced taxonomy beats free-form pre-mortem. The free-form prompt (“imagine the task has failed in 30 days; list 10+ causes”) is necessary but not sufficient. The MAST 14-mode interrogation surfaces categories the free-form pass clusters away from.

Prompt isolation for the validator is non-negotiable. Validators that share context with planners reproduce the planner’s confidence on shared content. The convergent literature on intrinsic self-correction limits, error-localization failure, and judge self-enhancement bias all point at this; we found it true in practice on the cases we inspected.

File-based handoffs survive context pressure; transcript-based handoffs do not. ASP

commits charter, macro plan, deliverables register and goal spec to disk before the executor is spawned. A child subprocess that reads from disk does not depend on parent transcript that may have been compressed or truncated.

Hard runtime constraints (Iron Laws) make a skill enforceable. Skills that read as “guidelines” are routinely ignored under context pressure. Skills that emit a checklist of non-negotiables — and whose templates *refuse to be filled in* without each box ticked — are followed.

Treating claude -p exit codes as semantic-success signals is a silent footgun. This is the finding reported in Flores (2026b); mentioning it here because the design implication (Iron Laws 11 and 12) was material to the system’s reliability.

7. Threats to Validity

We treat threats explicitly across the four categories standard in empirical software-engineering reports.

Construct validity. The same researcher who designed the protocol authored the expected-shadow-points lists *and* hand-adjudicated coverage. Even with the expected lists committed ex-ante and a written rubric, this is the single largest threat to the observed effect’s interpretability. The deterministic adjudicator we ship partially mitigates this by allowing any reviewer to re-adjudicate the recorded transcripts; it does not address the prior question of whether the expected lists themselves are biased toward what ASP surfaces. **Blind re-authoring by independent reviewers is the strongest follow-up.**

Internal validity. $n=8$ subagent dispatches is sufficient to demonstrate existence and approximate magnitude; it is *not* powered for causal estimation. No statistical test is reported. No formal ablation of components has been run (the qualitative attribution of load-bearing-ness to the MAST checklist in §5.2 is a hypothesis). The acceptance threshold of 80% is the author’s choice and not derived from prior literature.

External validity. Single underlying model (Claude Opus 4.7), single environment (macOS 14.5, Claude Code 2.1.143), five engineering tasks drawn from the author’s domain experience. We make no claim of generalisation beyond these conditions.

Reliability. All evaluation INPUT statements, expected-shadow-point lists, transcripts, and the deterministic-adjudicator code are committed to the companion repository under explicit git tags. The benchmark package builds a cryptographically-chained manifest per run so any byte edited under a run directory invalidates the manifest hash.

8. Limitations and Future Work

The limitations of this experience report match the threats above one-to-one. The highest-priority follow-ups, in order, are:

1. **Blind re-authoring** of the expected-shadow-point lists by 2+ independent reviewers with no exposure to ASP, plus blind adjudication of the recorded transcripts. Compute Cohen’s κ for inter-adjudicator agreement; re-measure coverage.

2. **Formal ablation** of the load-bearing hypothesis: MAST checklist on/off (testing §5.2), validator prompt isolation on/off, Phase 0a intake on/off. Each requires $n \geq 40$ per condition to power statistically.
 3. **Iron Law 11 empirical fill-in.** The companion paper Flores (2026b) specifies the $N=50$ confusion-matrix design as a pre-registered protocol; the empirical fill-in is the natural next step.
 4. **Cross-model evaluation.** Re-run on Claude Sonnet, GPT-5, Gemini 2.5, and an open-weight model.
 5. **Head-to-head against PreFlect.** Run ASP and a PreFlect reimplementaion on the same task set.
 6. **Embedding-based adjudicator** as a v0.2 of the benchmark, with pinned sentence-transformer model, to raise the recall floor on lexically-disjoint paraphrases.
 7. **Native-speaker review** of the ES, IT, and HE documentation translations.
 8. **Submission target.** Workshop venues — NeurIPS LLM-agents track, ICLR agents workshop, AAAI agents track — are the appropriate fit for the current empirical scale. Main-track submission requires the follow-ups above.
-

9. Conclusion

We described ASP, an operational pre-mortem skill for LLM coding agents, integrating Mitchell, Russo and Pennington’s (1989) prospective hindsight finding, Klein’s (2007) pre-mortem operationalisation, and Cemri et al.’s (2025) MAST 14-mode failure taxonomy as runtime discipline under a contract of twelve Iron Laws. The skill ships open source with three install paths. On five preliminary engineering evaluations, shadow-point coverage rose from approximately 47% to approximately 100% on the chosen tasks; the sample size, single-model condition, and non-blind adjudication make this preliminary rather than robust. The deterministic adjudicator shipped alongside the skill allows any reviewer to re-adjudicate the recorded transcripts without trusting the author.

We additionally observed that `claude -p` returns exit code 0 even when the agent gracefully refuses the requested goal — a quietly load-bearing finding for anyone wrapping the CLI in a shell pipeline. Because the observation merits independent characterisation at larger N , we report it as a pre-registered protocol in the companion paper Flores (2026b).

The strongest follow-up is blind re-adjudication. The most useful companion artefact is the $N=50$ confusion-matrix fill-in described in the companion paper.

Acknowledgments

This experience report was prepared in explicit human-in-the-loop collaboration with Claude (Anthropic, Opus 4.7), used as a drafting, verification, and review partner. Methodology decisions, empirical design, and finalisation were performed by the human author. The collaboration model is documented in the companion repository’s `docs/ARCHITECTURE.md` Parts 5–6.

The author thanks the maintainers of the [obra/superpowers](#) skill framework, the contributors of the open [agentskills.io](#) standard, and the Anthropic Claude Code team for the `/goal` primitive whose empirical behavior motivated Iron Laws 11 and 12.

References (APA, 7th edition)

- Anthropic. (2025). *Claude’s extended thinking* [Research announcement]. <https://www.anthropic.com/research/vis-extended-thinking>
- Anthropic. (2026a). *Claude Code* [Software documentation]. <https://code.claude.com>
- Anthropic. (2026b). *Agent Skills best practices* [Software documentation]. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices>
- Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). *Why do multi-agent LLM systems fail?* arXiv. <https://arxiv.org/abs/2503.13657>
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). *Improving factuality and reasoning in language models through multiagent debate*. ICML 2024. <https://arxiv.org/abs/2305.14325>
- Erdogan, L. E., Lee, N., Kim, S., Moon, S., Furuta, H., Anumanchipalli, G., Keutzer, K., & Gholami, A. (2025). *Plan-and-Act: Improving planning of agents for long-horizon tasks*. ICML 2025. <https://arxiv.org/abs/2503.09572>
- Flores, U. (2026b). *Graceful refusals as silent successes: A pre-registered protocol for characterising `claude -p` exit-code semantics*. See `paper/iron-law-11.md` in the companion repository.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). *Large language models cannot self-correct reasoning yet*. ICLR 2024. <https://arxiv.org/abs/2310.01798>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can language models resolve real-world GitHub issues?* ICLR 2024. <https://arxiv.org/abs/2310.06770>
- Kamoi, R., Zhang, Y., Zhang, N., Han, J., & Zhang, R. (2024). When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*. <https://arxiv.org/abs/2406.01297>
- Klein, G. (2007). Performing a project premortem. *Harvard Business Review*, 85(9), 18–19.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). *Self-Refine: Iterative refinement with self-feedback*. NeurIPS 2023. <https://arxiv.org/abs/2303.17651>
- Mitchell, D. J., Russo, J. E., & Pennington, N. (1989). Back to the future: Temporal perspective in the explanation of events. *Journal of Behavioral Decision Making*, 2(1), 25–38.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language agents with verbal reinforcement learning*. NeurIPS 2023. <https://arxiv.org/abs/2303.11366>
- Tyen, G., Mansoor, H., Cărbune, V., Chen, P., & Mak, T. (2024). *LLMs cannot find reasoning errors, but can correct them given the error location*. arXiv. <https://arxiv.org/abs/2311.08516>
- Wang, H., Cao, Y., Lin, L., & Chen, J. (2026). *PreFlect: From retrospective to prospective reflection in large language model agents*. arXiv. <https://arxiv.org/abs/2602.07187>

- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*. arXiv. <https://arxiv.org/abs/2308.08155>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. ICLR 2023. <https://arxiv.org/abs/2210.03629>
- Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., & Wu, Q. (2025). *Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems*. ICML 2025 Spotlight. <https://arxiv.org/abs/2505.00212>
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023 D&B. <https://arxiv.org/abs/2306.05685>
- Zhu, K., Liu, Z., Li, B., Tian, M., Yang, Y., Zhang, J., Han, P., Xie, Q., Cui, F., Zhang, W., Ma, X., Yu, X., Ramesh, G., Wu, J., Liu, Z., Lu, P., Zou, J., & You, J. (2025). *Where LLM agents fail and how they can learn from failures*. arXiv. <https://arxiv.org/abs/2509.25370>

Appendix A — Citation verification status

Every claim in the body cites at most a primary source. The table below records the verification status of each cited reference at the time of preprint submission. The tag scheme is the author’s own and is provided to make verification trivial for a reviewer; tags do not appear in the body for readability.

Tag	Meaning
[V]	Verified directly via the arXiv/journal/HBR URL listed above.
[D]	Anthropic documentation, treated as ground truth for documented tool behaviour.
[E]	Empirical observation from this work or the companion paper.
[O]	Author’s operationalisation; not present verbatim in any cited paper.

All references in the References section were verified by direct URL fetch on 2026-05-17. The MAST 14-mode names are quoted verbatim from Cemri et al. (2025) and listed in the companion skill file `skills/anticipating-shadow-points/mast-checklist.md`. Mitchell et al. (1989) is the primary source for the $\approx 30\%$ effect of prospective hindsight; Klein (2007) is the operational recipe.

Appendix B — Evaluation set and reproducibility

The five engineering evaluations are committed in the companion repository:

- `skills/anticipating-shadow-points/evals/01-supabase-migration.md` (E1)
- `skills/anticipating-shadow-points/evals/02-edge-function-deploy.md` (E2)

- `skills/anticipating-shadow-points/evals/03-refactor-shared-util.md` (E3)
- `skills/anticipating-shadow-points/evals/04-rls-policy-change.md` (E4)
- `skills/anticipating-shadow-points/evals/05-cron-skill-conflict.md` (E5)

Each file contains the INPUT statement, the EXPECTED shadow-points list, and the ACCEPTANCE threshold, all authored ex-ante. The benchmark package in `benchmark/` provides:

- a deterministic 4-layer adjudicator (exact / synonym / Jaccard / unmatched);
- per-run cryptographic provenance (env, code, inputs, outputs and adjudication hashed into a Merkle-style chain);
- a `verify` subcommand that re-walks a run directory and reports any byte-level drift;
- 27 unit tests pinning the determinism contract.

A reviewer can reproduce the adjudication scores by cloning the repository, installing the benchmark package, and running `asp-benchmark run --mode real --model claude-opus-4-7` followed by `asp-benchmark verify <run-id>`. The protocol is specified independently of the Python implementation in `benchmark/PROTOCOL.md` so it can be re-implemented in another language.

About the Author

Ulisses Flores is CTO and Chief Researcher at Codex Hash Research Laboratory, an independent R&D lab in São Paulo, Brazil. He builds production systems where silent failure is expensive — historically in algorithmic trading, embedded IoT and edge cryptography, and currently in agentic-AI tooling. The ASP skill described in this whitepaper grew out of that latter line of work; the empirical finding reported in the companion whitepaper grew out of using Anthropic’s `claude -p` runner as an autonomous execution kernel and watching it return exit code 0 on a graceful agent refusal.

He is an MSc candidate in Artificial Intelligence at American Global Tech University and writes in Portuguese, English, Spanish, and Italian.

Contact · `c.ulisses@gmail.com` · <https://ulissesflores.com>

© 2026 Ulisses Flores · Codex Hash Research Laboratory · São Paulo, Brazil Paper text licensed under CC BY 4.0 · Companion software licensed under MIT

End of whitepaper. References: 20 primary entries (1989, 2007 + 18 papers from 2022–2026) + 3 documentation entries. Word count: approximately 5,000 words excluding tables, code blocks, and appendices.